Министерство цифрового развития, связи и массовых коммуникаций
Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования «Сибирский государственный университет
телекоммуникаций и информатики»
(СибГУТИ)

Кафедра прикладной математики и кибернетики

Программирование графических процессоров

Лабораторная работа №3

Выполнили: Студенты 3-го курса,
группы ИП-111
Гердележов Даниил Дмитриевич
Махаева Алина Витальевна
Клепче Георгий Васильевич

Проверил преподаватель:
Малков Евгений Александрович

Новосибирск, 2024

Оглавление

Задание: 3
Цель: 3
Ход работы: 4
Вывод: 5
Листинг 6
main.cu 6

Задание:

1. Напишите программу сложения двух векторов на GPU.
2. Зафиксируйте длину векторов равной 1<<20.
3. Проведите серию запусков программы варьируя количество нитей в блоке, 1, 16, 32, 64,128, …, 1024, для запуска ядра.
4. Определите зависимость времени выполнения ядра, вычисляющего сумму векторов, от конфигурации нитей. Используйте для определения времени события CUDA и профилировщики.

Цель:

Априорное понимание влияния конфигурации нитей на производительность выполнения кода на GPU

Ход работы:

Device 0: NVIDIA GeForce GTX 1060 6GB
  Total global memory: 2147221504 bytes
  Total constant memory: 65536 bytes
  Shared memory per block: 49152 bytes
  Warp size: 32
  Max threads per block: 1024
  Max threads per multiprocessor: 2048
  Max grid size: (2147483647, 65535, 65535)
  Max block size: (1024, 1024, 64)
  Clock rate: 1771500 kHz
  Memory clock rate: 4004000 kHz
  Memory bus width: 192 bits

Напишем программу сложения векторов на gpu, установим длину векторов равную: 1<<20 проведем сложение векторов на различных конфигурациях - варьируя количество нитей в блоке, 1, 16, 32, 64,128, …, 1024. Засечем время для каждого сложения, после чего построим график зависимости времени выполнения операции от количества нитей в блоке (код программы приведен в листинге).

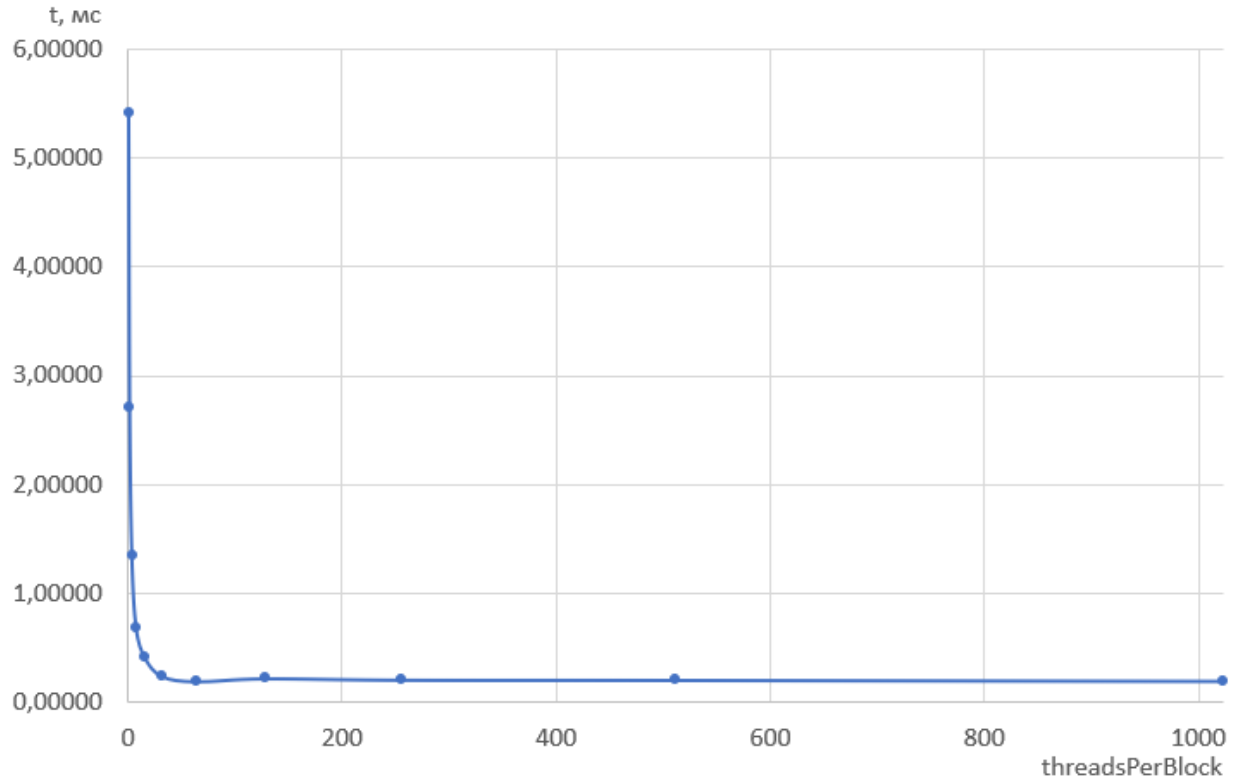


Рис. 1 - график зависимости времени выполнения операции сложения векторов от количества нитей в блоке.

По полученным нами данными можем сформировать следующую зависимость: чем больше нитей в блоке, тем быстрее происходит сложение векторов.

При увеличении количества нитей в блоке:

- с 1 до 2, с 2 до 4, с 4 до 8, с 8 до 16 наблюдалось двукратное уменьшение затрат времени;
- с 16 до 32, с 32 до 64 – полуторное;
- при дальнейшем увеличении количества нитей в блоке сокращение времени было незначительным.

Таким образом увеличение количества нитей с 1 до 64 нитей дало сокращение времени в 27 раз, с 64 до 1024 в 1.01.

Вывод:

- Мы написали программу сложения двух векторов на GPU, зафиксировали длину векторов равной 1<<20 и провели серию запусков программы варьируя количество нитей в блоке, 1, 16, 32, 64,128, …, 1024, для запуска ядра (код программы приведен в листинге).
- Определили зависимость времени выполнения ядра, вычисляющего сумму векторов, от конфигурации нитей.
- Изучили влияние конфигурации нитей на производительность выполнения кода на GPU (увеличение количества нитей с 1 до 64 нитей дало сокращение времени в 27 раз, с 64 до 1024 в 1.01).

Листинг

main.cu

```
#include <stdio.h>
#include <stdlib.h>
#include <cuda_runtime.h>

#define N (1 << 20)

__global__ void vectorAdd(int* a, int* b, int* c) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    if (i < N)
        c[i] = a[i] + b[i];
}

int main() {
    int* a, * b, * c;
    int* d_a, * d_b, * d_c;

    a = (int*)malloc(N * sizeof(int));
    b = (int*)malloc(N * sizeof(int));
    c = (int*)malloc(N * sizeof(int));

    cudaMalloc((void**)&d_a, N * sizeof(int));
    cudaMalloc((void**)&d_b, N * sizeof(int));
    cudaMalloc((void**)&d_c, N * sizeof(int));

    for (int i = 0; i < N; i++) {
        a[i] = i;
        b[i] = i * 2;
    }

    cudaMemcpy(d_a, a, N * sizeof(int), cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, N * sizeof(int), cudaMemcpyHostToDevice);

    int threadsPerBlock;

    for (threadsPerBlock = 1; threadsPerBlock <= 1024; threadsPerBlock *= 2) {
        int numBlocks = (N + threadsPerBlock - 1) / threadsPerBlock;

        cudaEvent_t start, stop;
        cudaEventCreate(&start);
        cudaEventCreate(&stop);

        cudaEventRecord(start);

        vectorAdd <<< numBlocks, threadsPerBlock >>> (d_a, d_b, d_c);
        cudaDeviceSynchronize();

        cudaEventRecord(stop);
        cudaEventSynchronize(stop);

        float milliseconds = 0;
        cudaEventElapsedTime(&milliseconds, start, stop);

        cudaMemcpy(c, d_c, N * sizeof(int), cudaMemcpyDeviceToHost);

        printf("%d threads per block: Time: %.5f ms\n", threadsPerBlock,
milliseconds);
        for (int i = 0; i < 10; i++) {
            printf("%d + %d = %d\n", a[i], b[i], c[i]);
        }
        printf("\n");
```

```
            cudaEventDestroy(start);
            cudaEventDestroy(stop);
        }

        cudaFree(d_a);
        cudaFree(d_b);
        cudaFree(d_c);

        free(a);
        free(b);
        free(c);

        return 0;
    }
```